

Modular Discrete-Time Simulation Framework

A Modular Framework for Signal Processing and Control Prototyping

Abstract

Building a modular discrete-time simulation engine provides a structures foundation for analysis dynamic systems through interconnected functional blocks. The architecture adopts a block-based modelling approach commonly used in control engineering, enabling clear representation of signal transformations. Core components compose the basis of simulation engine and reproduce essential behaviours found in discrete-time control systems.

The framework is evaluated through three brief case studies that examine the system response under different configurations. These experiments allow the analysis of transient dynamics, noise sensitivity, and the effectiveness of filtering techniques. Results obtained after simulation demonstrate that the engine summarises expected behaviours and provides insight into numerical stability, components interaction, and signal evolution.

In addition to analytical evaluation, the work highlights the educational and prototyping value of building a simulation framework from first principles,

enabling users to better understand the internal mechanisms and performance of control systems and enhance fundamentals of dynamic modelling.

1. Introduction

This project focuses on developing a modular discrete-time simulation engine for dynamic systems used in control engineering. This framework is built applying a block-based architecture to represent signal transformations, dynamic elements, and system interactions. Such architectures are essential in mechanical engineering industry, where control systems play a key role in automation, robotics, automotive, and physical processes modelling.

Beyond this practical purpose, the project also emphasises the educational and prototyping value of building a simulation engine from scratch, providing insight into the internal mechanisms of numerical integration, signal propagation, and block execution. This simulation engine is intentionally designed to be extensible, enabling future integration of more advanced features and additional functional blocks to allow the development of more complex control and signal-processing applications.

2. Objectives

The main objective of this project is to develop a discrete-time simulation engine capable of executing interconnected functional blocks and reproducing the behaviour of dynamic systems. To achieve this, the system applies the a set of core blocks, which enable the signal generation, mathematical operations, transformation, accumulation, and analysis.

This project also aims to validate the behaviour and reliability of the simulation engine through representative case studies, which evaluate the system response under noise injection, digital filtering, and step excitation. This allows the user to evaluate the numerical stability, accuracy, and signal characteristics produced by the engine, whilst ensuring consistent behaviour of the discrete-time implementation according to theoretical background.

3. Methodology

3.1 Theoretical Background

Discrete-Time Dynamic Systems

Dynamic systems are commonly modelled using continuous-time differential equations that describe how system states evolve in response to inputs. To implement these models on digital platforms, the continuous equations must be converted into discrete-time forms. This process allows the system behaviour to be evaluated at specific time intervals rather than continuously.

A fundamental numerical method used for this conversion is the **Forward Euler method**, which approximates the next state using the current state and its time derivative. In discrete form, this takes the structure

$$x_{k+1} = x_k + dt \cdot f(x_k, u_k) \quad \text{Eq. 1}$$

where x_k is the state, u_k the input, and Δt the chosen time step. This highlights the core modelling concepts of **state**, **input**, and **output**, each of which plays a distinct role in describing system behaviour.

Time discretisation is essential for modern digital control systems because controllers operate on sampled data rather than continuous signals. Choosing an appropriate sampling interval directly affects numerical accuracy, stability, and

real-time performance, making discretisation a critical step in transitioning from theoretical models to practical digital implementations.

Functional Blocks

The simulation framework is composed of several functional blocks that represent fundamental operations commonly used in control engineering and discrete-time dynamic modelling.

A typical control-software environment relies on a set of core functional blocks that together enable signal generation, manipulation, dynamic modelling, and feedback control. The **Step block** (Eq. 2) provides a sudden change in the input signal, allowing the system's transient and steady-state behaviour to be evaluated. Signals can then be scaled using a **Gain block** (Eq. 4), while an **Integrator** (Eq. 3) captures accumulation and first-order dynamics in discrete time. A **Constant block** (Eq. 5) supplies fixed reference values, and a **Scope** (Eq. 6) records signal evolution for visual analysis.

$$u_{\text{step}}(t) = \begin{cases} u_0 & \text{if } t < t_s, \\ u_1 & \text{if } t \geq t_s \end{cases} \quad \text{Eq. 2}$$

$$y_{k+1} = y_k + dt \cdot u_k \quad \text{Eq. 3}$$

$$u_{\text{gain}}(t) = K \cdot u(t) \quad \text{Eq. 4}$$

$$u(t) = c \quad \text{Eq. 5}$$

$$\{ u_0, u_1, u_2, \dots, u_k \} \quad \text{Eq. 6}$$

To support feedback and closed-loop behaviour, a **sum block** combines multiple signals, most commonly to compute the error between a reference and the measured output. This error is then processed by a **PID controller**, which

applies proportional, integral, and derivative actions to regulate system behaviour. In discrete-time implementations, the **unit delay** is essential for modelling sample-to-sample state propagation, ensuring that digital controllers and dynamic systems behave correctly under time discretisation.

Noise Disturbances Filtering

In many control and signal-processing applications, real-world measurements contain unavoidable disturbances that must be modelled and managed. Noise is typically represented as a **random perturbation added to an underlying base signal**, allowing simulations to capture the variability and uncertainty present in physical systems.

$$x_k = s_k + n_k \quad (\text{Eq. 6})$$

Where s_k is the underlying signal and n_k is a random disturbance. A common choice is uniform noise,

$$n_k = \text{Uniform}(a, b)$$

Which introduces bounded variability suitable for testing robustness in discrete-time systems.

To shape or reduce these unwanted components, digital filters are introduced. A **Low-Pass Filter** (Eq. 7) attenuates high-frequency content, producing a smoother signal that preserves slow system dynamics while suppressing rapid fluctuations.

$$y_{k+1}^{\text{LPF}} = y_k^{\text{LPF}} + \alpha_{\text{LPF}} \left(x_k - y_k^{\text{LPF}} \right) \quad \text{Eq. 7}$$

In contrast, a **High-Pass Filter** (Eq. 8) isolates fast variations, making it useful for highlighting noise, detecting abrupt changes, or analysing rapid system

behaviour. Together, these filtering tools play a crucial role in **digital signal processing** and control systems, improving measurement quality, enhancing controller performance, and ensuring robust operation in noisy environments.

$$y_{k+1}^{\text{HPF}} = \alpha_{\text{HPF}} \left(y_k^{\text{HPF}} + x_{k+1} - x_k \right) = x_k - y_k^{\text{HPF}} \quad \text{Eq. 8}$$

Digital filters play an essential role in signal processing and control engineering, enabling noise reduction, improved stability, and analysis of system behaviour under realistic noisy environments.

Simulation Execution

During each discrete-time step, the simulation engine evaluates all functional blocks sequentially, following the predefined execution order. At iteration k , each block computes its output according to its internal state and the current input signal. Once computed, signals are propagated through the block interconnections, ensuring that each component receives the correct input for the next evaluation cycle. State updates are performed according to block equations (Eq. 2, 3, 4, and 5).

Throughout the simulation, output signals generated by each block are stored for later inspection and comparison, allowing detailed analysis of system behaviour, verification of block performance, and dynamic interaction within the block network.

3.2 Simulation Framework

The simulation is structured as a modular, block-based architecture that enables the construction and evaluation of discrete-time dynamic systems. Each block represents a specific mathematical operation, and the framework manages the execution, signal propagation, and state update over time.

At every time step k , the engine evaluates all blocks in a sequence, where each block receives its corresponding input signal, applies its internal state update, and produces an output according to its mathematical operation. These outputs are then propagated through connections between blocks, ensuring that downstream blocks receive the correct inputs for the next iteration.

Throughout the simulation, all relevant components are recorded by the Scope block, which stores the evolution of inputs, intermediate values, and outputs. These data are collected for further analysis of case studies and verification of block behaviour.

4. Case Studies

4.1 Basic Dynamic Response to a Step Input

The first case study aims to validate that the tool is capable of performing the basics of control engineering, ensuring that the simulation engine behaves consistently reliably with simple theoretical expectations and confirming that the core computational logic is solid.

The configuration for this system consists of four interconnected components. A **Step** signal that transitions from 0 to 1 at a predefined time of $t = 1.0$, a **Gain** block with a constant factor of 2.0, after which the resulting value is

integrated by an **Integrator** initialised at 0. The output of the system is captured by a **Scope** block, providing a complete record of the for subsequent analysis.

$$u_{\text{step}}(t) = \begin{cases} 0, & t < 1 \\ 1, & t \geq 1 \end{cases}$$

$$u_{\text{gain}}(t) = 2 \cdot u_{\text{step}}(t)$$

$$y_{k+1} = y_k + dt \cdot u_{\text{gain},k}$$

The blocks are connected sequentially to define the signal flow through the system. The Step input provides a simple excitation that allows the dynamic behaviour of the system to be evaluated under predictable conditions, revealing how the system responds to sudden variations. The output of the Step block is connected to the Gain block, which applies a constant scaling factor to amplify the signal in a controlled manner and mitigate the abrupt variations of the Step block. The amplified signal is transferred to the Integrator, which accumulates the input over time introducing first-order dynamic behaviour in the system. This block aims to validate the core numerical logic of the simulation engine by accumulating previous values. Finally, the output of the integrator is connected to the Scope block, which records the signal variations throughout the simulation and allows the user to visualise the signals behaviour over time for sequential analysis.

$$y_k = 0.2 \cdot (k - 10)$$

4.2 Stable First-Order System Under Step Excitation

The second case study evaluates the behaviour of a simple first-order system in discrete time with the purpose of confirming that the simulation engine can reproduce stable dynamics under a constant input. Unlike the previous scenario, this configuration is designed to demonstrate the smooth convergence of the system when driven by a unit step and scaled by a unit gain.

The configuration of the system consists of four blocks; a Step input that produces a transition from 0 to 1 at $t = 1.0s$. Its output is connected to a Gain block with 1.0 as constant factor, ensuring that the integrator receives a constant input once the step occurs. An Integrator block initialised to 0 accumulates this input over time and models the behaviour of a stable first-order system. Finally, the Scope captures the output for further analysis. In this configuration, blocks are connected sequentially to form a linear forward path to isolate the essential dynamic behaviour of the integrator, which allows the simulation engine to be evaluated without noise interventions.

$$u(t) = c$$

$$y_{k+1} = y_k + dt \cdot c$$

Before the step occurs, the integrator output is zero. Once the step is applied, the integrator starts accumulating a constant value of 1 over time, producing a linear ramp with a slope equal to the time step.

$$y(t) = c \cdot t$$

4.3 Noise Injection and Digital Filtering

This case study evaluates the behaviour of the simulation framework when processing noisy signals at every simulation step through a sequence of digital filters. The objective is to validate that the tool can correctly propagate noise disturbances applying filtering operations in discrete time producing more stable outputs.

$$x_k = 1.0 + n_k, \quad n_k \sim \text{Uniform}(a, b)$$

The configuration starts with a base signal through a **Constant** block at a constant factor of 1.0. The output signal passes through a **Low-Pass-Filter** (LPF) with zero as initial value and $\alpha = 0.5$, allowing low-frequency signals to pass through, whilst high-frequency noise is blocked. Then, the output signal passes through the **High-Pass-Filter** (HPF) with $\alpha = 0.7$, allowing high-frequency noise disturbances to pass and finally, a **Scope** block to record the output data.

The noisy environment is injected by adding a uniformly distributed random perturbation to the base signal at every simulation step, which transforms the nominal value into a stochastic input. Therefore, the Low-Pass-Filter mitigates the rapid fluctuations produced by the noise, while the High-Pass-Filter attenuates the remaining fast variations.

5. Results

5.1 Basic Dynamic Response to a Step Input

TABLE 1. SIMULATION RESULTS OF CASE STUDY 1

Time [s]	Output y(t)
0.00	0.00
0.50	0.00
1.00	0.00
1.20	0.00
1.30	0.20
1.40	0.40
1.50	0.60
2.00	1.60
3.00	3.60
4.00	5.60
5.00	7.60
6.00	9.60
7.00	11.60
8.00	13.60
9.00	15.60
10.00	17.60

Table 1 shows the results obtained in case study 1, which consists on a basic system composed by one step, one gain, and one integrator. During the first second, the output value remains zero because the Step block did not change its value yet. When the step activates at $t = 1s$, the signal passes through the Gain,

which amplifies it using a constant factor of 2.0. Then the integrator starts to accumulate the value continuously.

From that moment on, the output increases linearly with a constant slope equal to the amplified value of the step. These results do not show oscillations, neither noise nor additional dynamics, as the integrator adds the input at each simulation step, producing a smooth and predictable evolution.

5.2 Stable First-Order System Under Step Excitation

TABLE 2. SIMULATION RESULTS CASE STUDY 2

Time [s]	Output $y(t)$
0.00	0.00
0.50	0.00
1.00	0.00
1.30	0.10
1.40	0.20
1.50	0.30
2.00	0.80
3.00	1.80
4.00	2.80
5.00	3.80
6.00	4.80
7.00	5.80
8.00	6.80
9.00	7.80
10.00	8.80
12.00	10.80
14.00	12.80
16.00	14.80
18.00	16.80
19.90	18.70

Table 2 shows the behaviour of a discrete integrator receiving a constant signal. The output starts from zero and increases linearly over time, reflecting the progressive accumulation of the input value at each simulation step.

The integrator does not introduce neither additional dynamics nor filtering, but adds the input value at each iteration. Therefore, the slope is constant and proportional to the value of the base signal. The result is a smooth, stable, and predictable evolution without oscillations and abrupt variations.

This case study confirms that the integrator performs correctly by accumulating the input signal and generating an output that increases uniformly over time.

5.3 Noise Injection and Digital Filtering

TABLE 3. SIMULATION RESULTS OF CASE STUDY 3

Time [s]	Output $y(t)$
0.00	0.000
0.20	0.350
0.3	0.679
1.00	0.321
2.00	-0.007
3.00	-0.382
4.00	-0.250
5.00	-0.002
6.00	-0.265
7.00	-0.210
8.00	-0.082
9.00	-0.336
10.00	-0.078

Table 3 shows a reduced set of representative values from the filtered signal. The high-pass filter responds strongly to fast variations in the noisy input, producing positive and negative peaks during the first seconds. As the signal evolves, the output oscillates around zero, showing the progressive removal of slow components and the preservation of rapid transitions.

6. Conclusion

In summary, the experiments conducted across the three case studies provide clear evidence of how the system behaves under different signal-processing conditions. The successful validation of the discrete-time simulation engine confirms that the execution cycle behaves consistently across all case studies, along with the correct operation of all fundamental blocks used in Control Engineering.

The system reliably demonstrates signal propagation between interconnected blocks, maintaining input–output relationships at every simulation stage. It is also confirmed that the Low-Pass Filter and High-Pass Filter respond appropriately to variations, showing realistic behaviour under noisy environments, as well as the capability of the system to handle random noise injection, which can be successfully attenuated.